

HW 5

Emma Johnson

February 28, 2025

13

```
weekly <- ISLR2::Weekly
```

e)

LDA

```
attach(weekly)
train <- (Year < 2009) # training data from 1990 - 2008
weekly.2009 <- weekly[!train, ] # held out data from 2009 - 2010
Direction.2009 <- Direction[!train]

# LDA Model:
lda.fit <- lda(
  Direction ~ Lag2,
  data = weekly,
  subset = train
)
lda.fit

## Call:
## lda(Direction ~ Lag2, data = weekly, subset = train)
##
## Prior probabilities of groups:
##      Down      Up
## 0.4477157 0.5522843
##
## Group means:
##      Lag2
## Down -0.03568254
## Up    0.26036581
##
## Coefficients of linear discriminants:
##      LD1
## Lag2 0.4414162

# Predictions using LDA model:
lda.class <- predict(lda.fit, weekly.2009)$class

# Confusion matrix:
table(lda.class, Direction.2009)
```

```
##           Direction.2009
## lda.class Down Up
##      Down    9  5
##      Up     34 56

# Fraction of correct predictions:
mean(lda.class == Direction.2009) # same as logistic

## [1] 0.625

# Test error:
mean(lda.class != Direction.2009)

## [1] 0.375
```

The LDA model of the *Weekly* data set predicts the correct *Direction* based on *Lag2* (for data in 2009 and 2010) 62.5% of the time. This is equivalent to the results from the logistic regression model.

f)

QDA

```
# QDA Model:
qda.fit <- qda(
  Direction ~ Lag2,
  data = weekly,
  subset = train
)
qda.fit

## Call:
## qda(Direction ~ Lag2, data = weekly, subset = train)
##
## Prior probabilities of groups:
##      Down      Up
## 0.4477157 0.5522843
##
## Group means:
##           Lag2
## Down -0.03568254
## Up    0.26036581

# Predictions using QDA model:
qda.class <- predict(qda.fit, weekly.2009)$class

# Confusion matrix:
table(qda.class, Direction.2009)

##           Direction.2009
## qda.class Down Up
##      Down    0  0
##      Up     43 61

# Fraction of correct predictions:
mean(qda.class == Direction.2009)

## [1] 0.5865385
```

```
# Test error:
mean(qda.class != Direction.2009)
```

```
## [1] 0.4134615
```

Under the QDA model, *Direction* is predicted correctly approximately 58.65% of the time.

g)

KNN with $k = 1$

```
library(class)
set.seed(5)

# Updating training & testing sets:
train.X <- matrix(Lag2[train], ncol = 1)
test.X <- matrix(Lag2[!train], ncol = 1)
train.class <- Direction[train]
test.class <- Direction[!train]

# Predictions using KNN:
knn.pred <- knn(train.X, test.X, train.class, k = 1)
mean(knn.pred != "Up") # 50% of observations classified "Up" and 50% "Down"
```

```
## [1] 0.4903846
```

```
# Confusion Matrix:
table(knn.pred, test.class)
```

```
##           test.class
## knn.pred Down Up
##      Down    21 30
##      Up     22 31
```

```
# Fraction of correct predictions:
mean(knn.pred == test.class)
```

```
## [1] 0.5
```

```
# Test Error:
mean(knn.pred != test.class)
```

```
## [1] 0.5
```

Under KNN with $k = 1$, *Direction* is predicted correctly 50% of the time, not any better than simply guessing.

h)

Naive Bayes

```
library(e1071)

# Naive Bayes Model:
nb.fit <- naiveBayes(
  Direction ~ Lag2,
  data = weekly,
  subset = train
)
nb.fit
```

```
##
## Naive Bayes Classifier for Discrete Predictors
##
## Call:
## naiveBayes.default(x = X, y = Y, laplace = laplace)
##
## A-priori probabilities:
## Y
##      Down      Up
## 0.4477157 0.5522843
##
## Conditional probabilities:
##      Lag2
## Y      [,1]      [,2]
## Down -0.03568254 2.199504
## Up    0.26036581 2.317485
```

```
# Predictions using Naive Bayes:
nb.class <- predict(nb.fit, weekly.2009)
```

```
# Confusion matrix:
table(nb.class, Direction.2009)
```

```
##      Direction.2009
## nb.class Down Up
##      Down    0  0
##      Up     43 61
```

```
# Fraction of correct predictions:
mean(nb.class == Direction.2009)
```

```
## [1] 0.5865385
```

```
# Test error:
mean(nb.class != Direction.2009)
```

```
## [1] 0.4134615
```

Under naive Bayes, *Direction* is predicted correctly approximately 58.65% of the time, similar to the QDA model.

i)

The method that provides the best results on the *weekly* data is either the logistic regression model or the LDA model, since their error rate and accuracy rates are equivalent.

14

```
auto <- ISLR2::Auto
auto$mpg01 <- ifelse(auto$mpg > median(auto$mpg), 1, 0)
```

c)

```
attach(auto)
```

```
## The following object is masked from package:lubridate:
```

```
##
##      origin
## The following object is masked from package:ggplot2:
##
##      mpg
set.seed(5) # reproducibility

# Sample to split 80% train & 20% test
train_ind <- sample(1:nrow(auto), size = 0.8 * nrow(auto))
train <- rep(FALSE, nrow(auto))
train[train_ind] <- TRUE # training set: 313 observations

test <- auto[!train, ] # held out data (testing): 79 observations
mpg01.test <- mpg01[!train] # test set mpg01 (true) values
```

e)

LDA

```
# LDA Model:
lda.fit <- lda(
  mpg01 ~ displacement + weight + year + horsepower,
  data = auto,
  subset = train
)
lda.fit

## Call:
## lda(mpg01 ~ displacement + weight + year + horsepower, data = auto,
##      subset = train)
##
## Prior probabilities of groups:
##      0      1
## 0.5303514 0.4696486
##
## Group means:
## displacement weight      year horsepower
## 0      271.3494 3611.602 74.46386 128.95783
## 1      115.1939 2346.707 77.65306  79.73469
##
## Coefficients of linear discriminants:
##                      LD1
## displacement -0.008804443
## weight      -0.001208599
## year         0.124503363
## horsepower    0.014875111

# Predictions using LDA model:
lda.class <- predict(lda.fit, test)$class

# Confusion matrix:
table(lda.class, mpg01.test)

##      mpg01.test
## lda.class  0  1
```

```
##           0 27  3
##           1  3 46

# Fraction of correct predictions:
mean(lda.class == mpg01.test) # training accuracy rate

## [1] 0.9240506

# Test error:
mean(lda.class != mpg01.test)

## [1] 0.07594937
```

The LDA model of the *Auto* data set, predicts *mpg01* (i.e. above or below the median mpg) based on *displacement*, *weight*, *year*, and *horsepower*, with a **test error** of approximately 7.59%. This is better than the results from the logistic regression model.

f)

QDA

```
# QDA Model:
qda.fit <- qda(
  mpg01 ~ displacement + weight + year + horsepower,
  data = auto,
  subset = train
)
qda.fit

## Call:
## qda(mpg01 ~ displacement + weight + year + horsepower, data = auto,
##      subset = train)
##
## Prior probabilities of groups:
##           0           1
## 0.5303514 0.4696486
##
## Group means:
## displacement weight year horsepower
## 0      271.3494 3611.602 74.46386 128.95783
## 1      115.1939 2346.707 77.65306  79.73469

# Predictions using QDA model:
qda.class <- predict(qda.fit, test)$class

# Confusion matrix:
table(qda.class, mpg01.test)

##           mpg01.test
## qda.class  0  1
##           0 27  3
##           1  3 46

# Fraction of correct predictions:
mean(qda.class == mpg01.test) # training accuracy rate

## [1] 0.9240506
```

```
# Test error:
mean(qda.class != mpg01.test)
```

```
## [1] 0.07594937
```

Under the QDA model, the **test error** is approximately 7.59%, similar to the results of the LDA model.

g)

Naive Bayes

```
# Naive Bayes Model:
nb.fit <- naiveBayes(
  mpg01 ~ displacement + weight + year + horsepower,
  data = auto,
  subset = train
)
nb.fit
```

```
##
## Naive Bayes Classifier for Discrete Predictors
##
## Call:
## naiveBayes.default(x = X, y = Y, laplace = laplace)
##
## A-priori probabilities:
## Y
##           0           1
## 0.5303514 0.4696486
##
## Conditional probabilities:
## displacement
## Y      [,1]      [,2]
## 0 271.3494 88.93742
## 1 115.1939 34.61993
##
## weight
## Y      [,1]      [,2]
## 0 3611.602 687.9543
## 1 2346.707 377.0782
##
## year
## Y      [,1]      [,2]
## 0 74.46386 2.961657
## 1 77.65306 3.706073
##
## horsepower
## Y      [,1]      [,2]
## 0 128.95783 36.85045
## 1 79.73469 15.48987
```

```
# Predictions using Naive Bayes:
nb.class <- predict(nb.fit, test)
```

```
# Confusion matrix:
table(nb.class, mpg01.test)
```

```
##          mpg01.test
## nb.class  0  1
##          0 26  3
##          1  4 46
```

```
# Fraction of correct predictions:
mean(nb.class == mpg01.test)
```

```
## [1] 0.9113924
```

```
# Test error:
mean(nb.class != mpg01.test)
```

```
## [1] 0.08860759
```

Under naive Bayes, the **test error** is approximately 8.86%.

h)

KNN

```
# Standardizing the quantitative variables:
standardized.X <- scale(auto[,c(3:5, 7)]) # displacement, horsepower, weight, year
apply(standardized.X, 2, mean) # means approx 0
```

```
## displacement horsepower weight year
## -5.786680e-17 -1.767977e-16 -8.020330e-18 -1.271467e-15
```

```
apply(standardized.X, 2, sd) # st. devs = 1
```

```
## displacement horsepower weight year
##          1          1          1          1
```

```
# Updating training & testing sets:
train.X <- standardized.X[train, ]
test.X <- standardized.X[!train, ]
train.class <- mpg01[train]
test.class <- mpg01[!train]
```

Testing $k = 1$:

```
set.seed(5)
```

```
# Predictions using KNN with k = 1:
knn.pred <- knn(train.X, test.X, train.class, k = 1)
mean(knn.pred == 1) # 60.76% observations classified as a 1 (i.e. above median mpg)
```

```
## [1] 0.6075949
```

```
# Confusion Matrix:
table(knn.pred, test.class)
```

```
##          test.class
## knn.pred  0  1
##          0 29  2
##          1  1 47
```

```
# Fraction of correct predictions:
mean(knn.pred == test.class)
```

```
## [1] 0.9620253
```



```

# Test Error:
mean(knn.pred != test.class)

## [1] 0.03797468

Testing k = 2:
set.seed(5)

# Predictions using KNN with k = 2:
knn.pred <- knn(train.X, test.X, train.class, k = 2)
mean(knn.pred == 1) # 54.43% observations classified as a 1 (i.e. above median mpg)

## [1] 0.5443038

# Confusion Matrix:
table(knn.pred, test.class)

##           test.class
## knn.pred  0  1
##           0 30  6
##           1  0 43

# Fraction of correct predictions:
mean(knn.pred == test.class)

## [1] 0.9240506

# Test Error:
mean(knn.pred != test.class)

## [1] 0.07594937

Testing k = 3:
set.seed(5)

# Predictions using KNN with k = 3:
knn.pred <- knn(train.X, test.X, train.class, k = 3)
mean(knn.pred == 1) # 56.96% observations classified as a 1 (i.e. above median mpg)

## [1] 0.5696203

# Confusion Matrix:
table(knn.pred, test.class)

##           test.class
## knn.pred  0  1
##           0 30  4
##           1  0 45

# Fraction of correct predictions:
mean(knn.pred == test.class)

## [1] 0.9493671

# Test Error:
mean(knn.pred != test.class)

## [1] 0.05063291

Testing k = 10:

```

```

set.seed(5)

# Predictions using KNN with k = 10:
knn.pred <- knn(train.X, test.X, train.class, k = 10)
mean(knn.pred == 1) # 59.49% observations classified 1 (i.e. above median mpg)

## [1] 0.5949367

# Confusion Matrix:
table(knn.pred, test.class)

##           test.class
## knn.pred  0  1
##           0 28  4
##           1  2 45

# Fraction of correct predictions:
mean(knn.pred == test.class)

## [1] 0.9240506

# Test Error:
mean(knn.pred != test.class)

## [1] 0.07594937

```

Under KNN, the **test error** is minimized when $k = 1$ at approximately 3.80%. Notably, the other trial with $k = 2, 3, 10$ each resulted in the same, if not lower, **test error** rate than the previous three methods, respectively.

Therefore, it appears that KNN is the best method for accurately predicting *mpg01*.